V

Document 2 of 3
Companion documents: User Manual · Business Rules & Validations

VISHAL MEGA MART · IT APPLICATIONS

Technical Logic Reference

Every piece of logic in the User Management System and the Asset Management System: what it does, how it works, and where it is implemented.

Applies to	UMS API · UMS Admin Portal · AMS.Api · AMS.Portal · BuildingBlocks
Audience	Developers, architects, support engineers, security reviewers
Version	1.0 · October 2026 · .NET 9, SQL Server 2019+

Contents

1 System overview

1.1 Solution layout · 1.2 Runtime components · 1.3 Design principles

2 Logic catalogue (index of all logics)

3 Identity, sign-in and sessions (UMS)

3.1 Sign-in · 3.2 Tokens · 3.3 Refresh and reuse detection · 3.4 Session validation · 3.5 Status, keep-alive, logout, revoke · 3.6 Lockout

4 Authorisation engine (UMS)

4.1 Decision layers · 4.2 Permission keys · 4.3 Role validity and scope · 4.4 Policies · 4.5 Conditions · 4.6 Schedules · 4.7 Endpoints · 4.8 UMS self-protection

5 UMS application pipeline

6 AMS web portal: authentication and session handling

7 AMS API: request handling

8 AMS use cases (business logic)

9 AMS data layer: Dapper and stored procedures

10 Notifications (e-mail)

11 In-house building blocks: VMM.Mediator, VMM.Mapper

12 Configuration reference

13 Tests

1 System overview

1.1 Solution layout (`UMS_Sol.sln`)

Project	Layer	Responsibility
`DomainLayer`	Domain	Entities (User, Tenant, Application, Role, Policy, Schedule, UserSession, LoginAttempt); value objects; the decision engine `AccessDecisionService` , `ConditionEvaluator` , `ScheduleEvaluator` .
`ApplicationLayer`	Application	Commands and queries per feature (`Features/AccessControl/*`), DTOs, FluentValidation validators, pipeline behaviours, ports (repository and service interfaces), mappings.
`Infrastructure`	Infrastructure	In-memory and SQL Server (Dapper) repositories, demo seed, password hashing, JWT issuing, session validator, audit service.
`WebApi`	Presentation	UMS REST API `/api/v1` , JWT bearer authentication, `RequirePermission` filter, exception logging middleware.
`UMS.AdminPortal`	Presentation	Blazor Server + MudBlazor admin console; calls the UMS API over HTTP.
`AMS.Domain`	Domain	AMS contracts (`BaseResponse`), read models, enums, exceptions, permission catalogue.
`AMS.Application`	Application	AMS use cases; ports for repositories, guard, API calls and e-mail; templates and recipient resolvers.
`AMS.Infrastructure`	Infrastructure	Dapper repositories, UMS client and guard, SMTP and notification queue, `ApiCallService` .
`AMS.Api`	Presentation	AMS REST API `POST api/v1/{action}` ; the only component touching the AMS database.
`AMS.Portal`	Presentation	Blazor Server portal; talks only to AMS.Api (data) and UMS (sign-in).
`VMM.Mediator` , `VMM.Mapper`	Building blocks	In-house mediator and object mapper (replace MediatR and AutoMapper).
`UMS.Tests` , `AMS.Tests` , `BuildingBlocks.Tests`	Tests	xUnit: 45 + 29 + 14 tests.

1.2 Runtime components

Browser
HTTP-only cookie; no tokens in the browser

AMS.Portal :5300
Blazor Server; holds UMS tokens server-side per session

AMS.Api :5310
validates UMS JWT; runs use cases

UMS API :5125
sign-in, sessions, decisions, directory

SQL Server
UMS DB · VMM_AMS DB (separate)

```
Browser ──cookie──► AMS.Portal ──IAPICall (POST api/v1/X, Bearer UMS token)──► AMS.Api ──► use
case ──► IAccessGuard ──► UMS /api/v1/me/authorize
                          │                                                              └──►
repository ──► Dapper ──► SQL Server (stored procedures)
                          └──► UMS /api/v1/auth/login | refresh | session | logout
Browser ──► UMS.AdminPortal ──► UMS API ──► VMM.Mediator handlers ──► repositories ──► SQL Server
/ in-memory
```

1.3 Design principles

- **Single decision point.** Every access decision is made by `AccessDecisionService` in UMS. AMS contains no role checks; it describes the action and the record to UMS and acts only on an Allow.
- **Fail closed.** An unreachable UMS, a missing attribute or an ended session never results in access.
- **Clean architecture / SOLID.** Dependencies point inwards (Presentation → Infrastructure → Application → Domain). Use cases depend on interfaces only. Each class has one reason to change (for example sender, template, resolver and recipient policy are separate e-mail classes).
- **Tenant isolation.** The tenant always comes from the signed-in user's token, never from a request body.
- **Explainability.** Every decision carries the step that decided it and a human-readable reason, shown to users and written to the user action trail.

2 Logic catalogue

Each logic has an identifier used throughout this document. "Where" is the main implementation.

ID	Logic	Where
L-01	Sign-in with lockout, account and application checks	`SessionFeatures.cs · LoginCommandHandler`
L-02	Single-device login (replace other sessions)	`LoginCommandHandler`
L-03	Session-bound access token issuing	`IdentityService · JwtTokenService`
L-04	Refresh-token rotation and reuse detection	`RefreshSessionCommandHandler · RefreshTokens`
L-05	Per-request session validation (10 s cache) and end recording	`Infrastructure/Services/SessionValidator.cs · WebApi/Program.cs`
L-06	Session status, keep-alive (touch), logout, admin revoke	`GetSessionStatusQuery · LogoutCommand · RevokeSessionCommand`
L-07	Login attempt counting and lockout	`DomainLayer/Entities/UserSession.cs · LoginAttempt`
L-08	Seven-layer access decision	`DomainLayer/Services/AccessDecisionService.cs`
L-09	Permission key matching with wildcards	`PermissionKey · Role.Grants`
L-10	Role validity dates and scope matching	`AccessDecisionService.ActiveAssignments · ScopeMismatch`
L-11	Policy targeting, priority and effects (Restrict / Deny / Grant)	`AccessDecisionService · PolicyApplies`
L-12	Condition evaluation and attribute context	`ConditionEvaluator · BuildContext`
L-13	Office hours, overnight windows and holidays	`ScheduleEvaluator`
L-14	Batch authorisation for applications (max 2000)	`CheckMyAccessQuery · MeController`
L-15	UMS protects its own API (RequirePermission)	`WebApi/Filters/RequirePermissionAttribute.cs`
L-16	Tenant resolution (X-Tenant-Id, super-admin cross-tenant)	`CurrentTenantService · controllers`
L-17	Request pipeline: validation behaviour, Result → HTTP	`ValidationBehavior · AccessControlControllerBase`
L-18	Audit trail, user action trail, exception log	`AuditService · ExceptionLoggingMiddleware`
L-19	Portal sign-in hand-off (ticket → cookie)	`AMS.Portal/Auth/AuthFlow.cs · LoginTicketStore`
L-20	Server-side token store and serialised refresh	`AMS.Portal/Auth/Sessions.cs · UmsTokenStore`
L-21	Cookie validation per HTTP request	`CookieValidator`
L-22	Circuit revalidation and server idle backstop	`AppRevalidatingAuthStateProvider`

ID	Logic	Where
L-23	Browser idle monitor, warning and keep-alive	`IdleTimeoutMonitor.razor · wwwroot/js/idle-timeout.js`
L-24	Antiforgery-protected sign-out and session-end routing	`AccountEndpoints · RedirectToLogin · Ui.SignOut`
L-25	API call service (IAPICall)	`AMS.Infrastructure/Api/ApiCallService.cs`
L-26	AMS JWT validation and current actor	`AMS.Api/Program.cs · HttpCurrentActor`
L-27	Exception → response mapping	`AMS.Api/Infrastructure/ApiPlumbing.cs`
L-28	UMS guard (fail closed) and network detection	`AMS.Infrastructure/Ums/UmsClient.cs · UmsAccessGuard`
L-29	List filtering and per-row action matrix	`GuardExtensions.FilterAsync / MatrixAsync`
L-30	Capabilities (menu and buttons)	`ReportUseCases.CapabilitiesAsync`
L-31	Record attributes sent to UMS	`AMS.Application/Common · ResourceAttrs`
L-32	Asset life cycle: create/edit (double check), check-out/in, audit, delete	`AssetUseCases`
L-33	Asset request workflow	`RequestUseCases`
L-34	Licence seats	`LicenseUseCases · PRO_AMS_LICENSE_*`
L-35	Stock: accessories, consumables, components	`StockUseCases · PRO_AMS_STOCK_*`
L-36	Reports: depreciation, due lists, dashboard	`ReportUseCases · Depreciation`
L-37	CSV export with formula-injection protection	`AMS.Application/Common · Csv`
L-38	Stored procedure contract and error codes	`DapperHelper · database/002_ams_procedures.sql`
L-39	Database initialisation and demo seed	`DatabaseInitializer · DemoDataSeeder`
L-40	Notification queue, notifier, templates, recipients, policy	`AMS.Infrastructure/Email · AMS.Application/Notifications`
L-41	Daily digest scheduling	`DailyDigestService · DailyDigestBuilder`
L-42	People directory with cache	`UmsPeopleDirectory`
L-43	Mediator: dispatch, pipeline, events	`BuildingBlocks/VMM.Mediator`
L-44	Object mapper: conventions, validation	`BuildingBlocks/VMM.Mapper`

3 Identity, sign-in and sessions (UMS)

3.1 Sign-in (L-01, L-02)

`POST /api/v1/auth/login` with `email, password, appCode, clientIp, userAgent`. The handler runs these steps in order; the first failure ends the request.

1. **Lockout check.** Load the `LoginAttempt` for the login name. If `LockedUntil` is in the future → `auth.locked` ("Try again after hh:mm").
2. **Password check.** Find the user by e-mail and verify the PBKDF2 hash. On failure, count the failure (L-07). If the count reaches `MaxFailedLogins` the login locks for `LockoutMinutes` → `auth.locked`, otherwise `auth.invalid`. Unknown e-mail and wrong password give the same answer (no account enumeration).
3. **Account check.** The tenant must be active and the user usable today (status Active, account expiry not passed) → otherwise `auth.disabled`.
4. **Application check.** `appCode` (default `UMS`) must be registered and enabled for the user's tenant → otherwise `auth.app`.
5. **Reset** the failure counter.
6. **Single-device login.** If `SingleDeviceLogin` is on, every open session of the user in the same application is ended with reason `Replaced`, and its validator cache entry is cleared.
7. **Create the session** (`UserSession`): id, tenant, user, emp code, app, IP, user agent (max 300 chars), `ExpiresAt = now + RefreshTokenDays`, `LastSeenAt = now`, refresh-token hash.
8. **Record** the last login and a user action log entry (`{APP}.SESSION.SIGN_IN`, "session on another device was ended" when L-02 applied).
9. **Issue** tokens (L-03) and return `LoginResultDto`: access token and expiry, refresh token, session expiry, session id, user id, tenant id, name, e-mail, emp code, app code, idle timeout minutes, `EndedOtherSessions`.

3.2 Access tokens (L-03)

HS256 JWT, lifetime `JwtExpiryMinutes` (default 15), issuer and audience from `Jwt:*`; the key must be at least 32 bytes (checked at start-up). Claims:

Claim	Meaning
`sub` / name identifier	User id
`sid`	Sign-in session id. Every request is checked against this session (L-05).
`app`	Application the session belongs to (AMS.Api accepts only `AMS`).
`tenant_id`, `emp_code`, name, e-mail	Identity of the caller, used for tenant isolation and audit.
`jti`	Unique token id.

3.3 Refresh and reuse detection (L-04)

A refresh token has the form `{sessionId:N}.{random}`; only its SHA-256 hash is stored. `POST /api/v1/auth/refresh`:

1. Parse the session id; unknown session → `session.expired`.
2. Compute the session status (ended, expired, idle). If not active → end it (if not already) and return `session.<reason>`.
3. If the presented token does not match the current hash, it is an **old token** (already rotated): someone may hold a copy, so the whole session ends with `TokenReuse`.
4. The user and tenant must still be usable → otherwise `AccountDisabled`.
5. Rotate: new refresh token, store its hash, clear the validator cache, issue a new access token.

3.4 Session validation on every request (L-05)

JWT bearer authentication in the UMS API has an `OnTokenValidated` hook:

- No `sid` claim → fail ("token is not bound to a sign-in session").
- `ISessionValidator.CheckAsync(sid)` loads the session and the tenant's idle timeout. The result is cached for **10 seconds** per session, and the cache entry is cleared whenever UMS ends or rotates the session.
- Status order: ended → its reason; `now ≥ ExpiresAt` → `Expired`; `now - LastSeenAt > idle` → `Idle`; otherwise active.
- If the session ran out but is not yet recorded as ended, the validator **records the end** (reason and time). The admin's Active Sessions page and the audit then show it.
- Not active → authentication fails; `OnChallenge` adds the header `X-Session-End: <Reason>`, which CORS exposes to browsers.

3.5 Status, keep-alive, logout and revoke (L-06)

Endpoint	Logic
`GET /auth/session?touch=`	Returns active / reason / expiry / last seen / idle minutes. With `touch=true` and an active session, `LastSeenAt = now` (user activity). A run-out session is ended and recorded.
`POST /auth/logout`	Ends the caller's own session with `SignedOut` or `Idle` (the only reasons a client may give).
`GET /sessions?activeOnly=`	Sessions of the tenant (permission `UMS.SETUP.SESSION.View`).
`DELETE /sessions/{id}`	Ends a session with `RevokedByAdmin`; audited as a Revoke (permission `UMS.SETUP.SESSION.Delete`).

3.6 Lockout (L-07)

`LoginAttempt(Login, FailedCount, LockedUntil)`. `RegisterFailure(now, max, lockout)` increments the count; when it reaches `max`, `LockedUntil = now + lockout` and the count resets. A successful sign-in clears it. Attempts are keyed by login name, so locking works for unknown names too.

4 Authorisation engine (UMS)

4.1 Decision layers (L-08)

`AccessDecisionService.Decide(request, context)` answers: *can user U perform action A on record R in module M of application P, now, from this network, device and channel?* Layers run in order. The first failing layer denies (deny-overrides), and every layer adds an explanation step.

#	Layer	Passes when	Example deny reason
1	Tenant	Tenant exists and is active	tenant ... is inactive
2	Identity	User active and not past account expiry	account expired on 2027-03-31
3	Application	App registered, active, enabled for the tenant, and some active role grants `APP.*.*.*` or more specific	no role grants access to Finance ERP
4	Module	A role grants something in the module	no permission on module Reports
5	Resource · Action	Resource exists, action valid for it, and a role grants the exact key	missing POS.BILLING.INVOICE.REFUND
6	Scope	At least one granting role assignment is in scope for the record	location: role scoped to 'HO-GGN', resource is 'S045'
7	Policies	Every applicable Restrict holds and no applicable Deny matches	Deny 'Supervisor refund limit ₹10,000' — resource.amount (15000) > '10000'

In layers 3–6 an applicable **Grant** policy whose conditions hold can stand in for a missing role permission (e.g. "employees see assets issued to them"). Restrict and Deny policies still apply afterwards.

4.2 Permission keys (L-09)

`APP.MODULE.RESOURCE.ACTION`, upper-case. Each segment may be `*`. A role grants a key if any of its patterns matches segment by segment: `AMS.*.*.*` grants every AMS action, `*.*.*.VIEW` every view. Actions: View, Create, Edit, Delete, Approve, Export, Print, Void, Refund, Checkout, Checkin, Audit, Request. A resource declares which actions are valid for it; an action the resource does not declare is denied at layer 5.

4.3 Role validity and scope (L-10)

- **Validity.** An assignment counts only if the role is active and `ValidFrom ≤ today ≤ ValidTo` (open-ended when empty). Example: Rahul's temporary Procurement Head role is ignored after 10 Oct 2026.
- **Scope.** An assignment may carry a scope, e.g. `{"location": "HO-GGN"}` or `{"store": "S045,S046"}` (comma = any of). For every scope key that the *record* carries, the record's value must be one of the listed values (case-insensitive). Keys the record does not carry do not restrict, so licences (no location) are visible to a location-scoped manager.
- If any granting assignment is in scope, layer 6 passes. If only scoped assignments grant the key and none is in scope, a Grant policy can still allow; otherwise deny with the mismatch.

4.4 Policies (L-11)

- A policy **targets** app / module / resource / actions (each may be `*`), optionally only users holding certain roles.
- Applicable policies are evaluated in **Priority** order (lower number first).
- **Restrict**: conditions must be true, else deny. **Deny**: if conditions are true, deny. **Grant**: if conditions are true, allow without a role permission.
- Conditions combine with **All** (AND) or **Any** (OR).

4.5 Conditions and attribute context (L-12)

The engine builds a context dictionary for each request:

Prefix	Attributes
`user.*`	id, empCode, department, designation, location, region, employmentType, mfaEnabled, roles (comma list) + custom user attributes (e.g. vendorCode)
`resource.*`	app, module, type + every attribute the application sends for the record (store, amount, cost, location, ownerEmpCode…)
`env.*`	date, time, day, ip, network, device, channel, isHoliday; `env.now` for schedule operators
`action`	The requested action

Operators: `= ≠ in "not in" > ≥ < ≤ contains "starts with" "within schedule" "outside schedule"`, all case-insensitive. Numbers are compared numerically, otherwise as text. The right-hand side may be a literal or another attribute (`resource.store = user.location`). A missing attribute never satisfies a condition, so a Restrict fails and a Deny or Grant does not match: **fail closed**.

4.6 Office hours and holidays (L-13)

A schedule has a time zone (Asia/Kolkata) and weekly windows such as Mon–Sat 09:30–18:30. A window whose end is before its start runs overnight (22:00–06:00: after-midnight times belong to the previous day's window). If the schedule excludes holidays, a holiday for the tenant closes it; holidays may be all-India or limited to regions, matched against the user's region.

4.7 Decision endpoints (L-14)

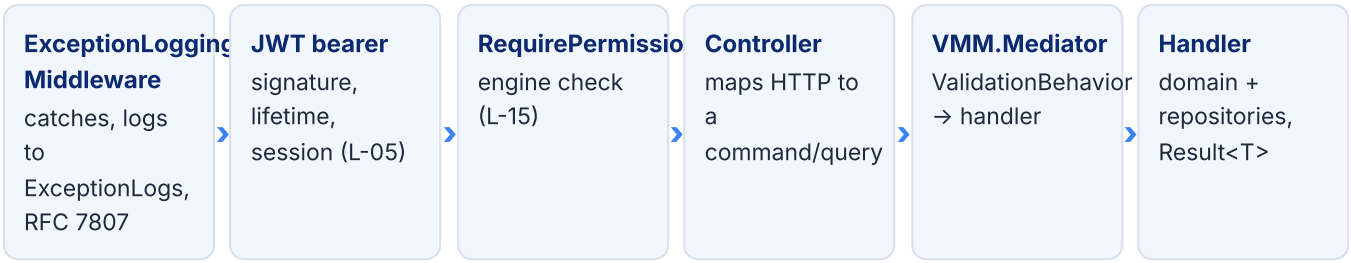
Endpoint	Use
`POST /authorize`	Decide for any user (service integrations, simulator). Permission `UMS.POLICIES.DECISION.VIEW`. Logged.
`POST /authorize/week`	The same request for every hour of a week (simulator heat-map).
`POST /me/authorize`	Decide for the caller (from their own token): AMS.Api uses this for every action. Logged to the user action trail.
`POST /me/authorize/batch`	Up to **2000** checks in one call, answers in order, not logged (used to filter lists and build menus).

Endpoint	Use
`GET /me/directory?search=`	Colleagues of the caller's tenant (AMS employee picker).

4.8 UMS protects itself (L-15, L-16)

- Each admin endpoint carries `[RequirePermission(module, resource, action)]`. The filter asks the same engine about the caller for `UMS.<module>.<resource>.<action>` and returns 403 with the reason when denied.
- `Security:RequireAuthentication = false` skips this filter. It is meant only for local demos and is **true** by default.
- The tenant of a request comes from the `X-Tenant-Id` header, else the `tenant_id` claim. Only an active `SUPER_ADMIN` may work in another tenant.

5 UMS application pipeline (L-17, L-18)



- **Validation.** `ValidationBehavior` runs every FluentValidation validator for the request; failures raise a validation exception, which the API returns as 400 with the field messages.
- **Result mapping.** Handlers return `Result<T>` with an error code and message. The controller base maps a failure to an RFC 7807 problem: sign-in, refresh and session failures (`auth.*` , `session.*`) → 401 with a `code` extension; every other failure → 400 with the message.
- **Audit trail.** Every create, update, delete and revoke writes an `AuditLogs` row (actor, entity, action, summary, before/after JSON, IP).
- **User action trail.** Single decisions and sign-ins write `UserActions` (user, app, route, action key, Allow/Deny, reason, IP).
- **Exception log.** Unhandled exceptions are stored with severity, path, user and stack trace; admins mark them resolved.

6 AMS web portal: authentication and session handling

6.1 Sign-in hand-off (L-19)

Blazor Server runs over a WebSocket, which cannot set cookies. The portal therefore signs in in two steps:

1. The sign-in page calls UMS `auth/login` (app `AMS`, browser IP and user agent) from the server.
2. On success the tokens go into the server-side **token store** (L-20), and a **single-use ticket** (valid 60 seconds, in memory) is created.
3. The page posts a hidden form with the ticket and an antiforgery token to `POST /account/complete-login`.
4. That endpoint redeems the ticket and writes the HTTP-only cookie: `__Host-AMS.Auth`, Secure, SameSite=Lax, 60 minutes sliding, 12 hours absolute. Then it redirects to the return URL (local URLs only).

The cookie contains only the identity and the session id; UMS tokens never reach the browser.

6.2 Token store (L-20)

- Holds per session id: access token and expiry, refresh token.
- An access token within **60 s** of expiry is refreshed through UMS. Refreshes are serialised per session (one refresh at a time, then double-checked), so parallel API calls do not trigger token-reuse detection.
- When UMS refuses a refresh, the reason is recorded in `EndedSessions` (kept 13 h) and the tokens are dropped.

6.3 Checks while the user works (L-21, L-22)

Check	When	Logic
Cookie validator	Every HTTP request	Quick check: absolute lifetime, an ended-session record, tokens present. At most every `RevalidationSeconds` it also asks UMS `auth/session`. If the session ended, the principal is rejected and the cookie removed. If UMS is unreachable the request passes and is checked again next time.
Circuit revalidation	Every `RevalidationSeconds` (60; 20 in dev) on open pages	Ends the session if an API call already learned it ended, if the server idle backstop is reached, or if UMS reports it ended. Tolerates up to 5 consecutive UMS failures before signing out.
API call result	Each API call	A 401 with `X-Session-End` records the reason and triggers sign-out with that reason.

6.4 Idle timeout (L-23)

- The timeout is the tenant's `SessionTimeoutMinutes`, carried in the sign-in.

- `idle-timeout.js` listens for pointer, key, wheel, scroll and touch events. It reports activity to the server at most every 10 s, and shares it between tabs through `localStorage`.
- While the user is active it calls `POST /account/keepalive` every timeout/4, clamped to 30 s–5 min. This renews the cookie and calls UMS `auth/session?touch=true`.
- The monitor ticks every second. At one minute left it shows the warning with a countdown; at zero it signs out with reason `idle`.

6.5 Sign-out and session-end routing (L-24)

- `POST /account/sign-out` requires an antiforgery token, so a link or image on another site cannot sign users out.
- It records the end, calls UMS `auth/logout` for `signedout` and `idle` (UMS already knows the other reasons), drops the tokens, clears the cookie, sends `Clear-Site-Data: "cache"`, and redirects to the sign-in page with `?reason=`.
- If a page becomes anonymous because the session ended, `RedirectToLogin` routes through the sign-out page, so the cookie is cleared and an idle session is closed in UMS.
- The sign-in page translates the reason into a message: replaced, idle, revokedbyadmin, tokenreuse, accountdisabled, expired, signedout.

7 AMS API: request handling

7.1 API call service (L-25)

The portal reaches the API only through `IAPICall.CallingAPI<TRes,TReq>(apiName, request)`:

- The route comes from `AmsApi:Routes[apiName]`, default `api/v1/{apiName}`. Names are constants in `AmsApiNames`, e.g. `GET_ASSETS` = `GetAssets`.
- It sends `POST` with a JSON body (property names as in C#, enums as strings), `Authorization: Bearer <fresh UMS token>` and `X-Client-IP`.
- A JSON `BaseResponse` is returned as is, for success and for refusals. A 401 also calls `SessionEnded(X-Session-End)`.
- It **never throws** to pages. Transport, timeout and JSON errors become `responseCode = 1004` with a message; no token gives 401 without calling.

7.2 JWT validation and current actor (L-26)

- **Validation.** AMS.Api validates the UMS token locally: issuer, audience, signing key, lifetime (30 s clock skew). It then requires `app == AMS` and a `sid`. A UMS-admin token is therefore rejected.
- **Current actor.** `HttpCurrentActor` builds the `Actor` (user id, tenant, emp code, name, e-mail) from the claims and keeps the bearer token for calls to UMS.
- **Client IP.** `X-Client-IP` is trusted only from configured forwarders (loopback plus `Ums:TrustedForwarders`). Otherwise the connection address is used.

7.3 Exception → response mapping (L-27)

Exception	HTTP	responseCode	Notes
AccessDeniedException	403	403	Message = UMS reason; logged at Information with the permission key
SessionEndedException	401	401	Adds `X-Session-End`
AuthorizationUnavailableException	503	503	UMS unreachable: fail closed
AmsException NotFound / Duplicate / other	404 / 409 / 422	1001 / 1003 / 1002	Business rule messages from use cases or stored procedures
Anything else	500	1004	Generic message; full error logged

8 AMS use cases (business logic)

8.1 Guard and filtering (L-28 – L-31)

- `UmsAccessGuard.EnsureAsync(check)` calls UMS `me/authorize` with the user's own token and the environment:
  - IP;
  - network: *Office LAN* for loopback and private addresses (10/8, 172.16/12, 192.168/16, IPv6 site and link local, fc00::/7), else *Internet*;
  - device and channel, from configuration.

  A deny throws `AccessDeniedException` with UMS's reason. UMS 401 becomes `SessionEndedException`; any other UMS failure becomes `AuthorizationUnavailableException` (fail closed).
- `FilterAsync` keeps the items whose View check UMS allows, in one batch call. `MatrixAsync` asks for N items × K actions in one batch and returns a bool grid; it fills the `Can*` flags of each row.
- **Capabilities** (`GetCapabilities`) are type-level checks of every module/resource/action, used for the menu and the page buttons. They only shape the UI; each action is checked again with the record's attributes.
- **Record attributes** (`ResourceAttrs`):
  - asset: assetTag, cost, requestable, location, region, category, status, statusType, department, assignedToEmpCode;
  - check-out adds assigneeEmpCode, assigneeEmploymentType, assigneeDepartment, assigneeLocation, assigneeRegion;
  - request adds requestedByEmpCode and requestStatus;
  - maintenance: cost (job) and assetCost;
  - stock: cost = unit cost × quantity, and quantity;
  - licence: cost, seats, category, manufacturer.

8.2 Asset life cycle (L-32)

Operation	Guard checks (UMS)	Rules (use case + stored procedure)
List / export / labels	View per asset (filter); Export or Print at type level	Filters: search, category, location, status, holder.
Get detail	View on the asset; row actions; Maintenance.Create	Includes history, maintenance, licences, components, current value.
Create	Create with the new record's attributes	Tag and name required; cost ≥ 0; tag unique per tenant; model, status and location must exist.
Edit	Edit on the record **as it is** and **as it will be**	Moving to a location outside one's scope is therefore denied.
Check out	Checkout with asset + assignee attributes	Employee required and active in the directory; return date not in the past; asset not already issued; status Deployable. Publishes the check-out e-mail.

Operation	Guard checks (UMS)	Rules (use case + stored procedure)
Check in	Checkin	Asset must be checked out; optional new status must exist. Publishes the check-in e-mail.
Audit	Audit	Next audit date must be in the future; sets last audit = today.
Delete	Delete	Asset must be checked in.

8.3 Asset request workflow (L-33)

Requestable assets: Requestable, Deployable, free, Request allowed, no own pending request › **Create** Request check (requestedBy = me); reason required; one pending per person and asset; e-mail desk › **Approve** Approve on request *and* Checkout on asset for the requester; issues the asset atomically › **Reject** Approve permission; reason required › **Cancel** Void permission; Pending only

Decisions are allowed only while the request is **Pending**. Approval issues the asset, marks the request Approved, and **rejects every other pending request for the same asset** ("Asset issued to ..."), all in one stored-procedure transaction. The requester receives the decision e-mail.

8.4 Licences (L-34) and stock (L-35)

- **Licence:**
  - seats ≥ 1, and seats cannot be reduced below the number in use;
  - a seat goes to a person and/or an asset; one seat per person per licence;
  - no check-out when all seats are used or the licence has expired;
  - deleting requires all seats checked in;
  - the product key is returned only when the caller may edit the licence.
- **Stock kinds** map to separate catalogue entries (ACCESSORIES / CONSUMABLES / COMPONENTS):
  - **Accessory:** to a person, one each, returnable.
  - **Consumable:** to a person, quantity ≤ available, reduces stock, not returnable.
  - **Component:** into an asset, returnable.

  Quantity cannot drop below the units out; delete requires all units in; the kind of an item cannot change. `Low` = available ≤ minimum quantity.

8.5 Reports (L-36) and CSV (L-37)

- **Depreciation** (straight line):
  - months = whole months since purchase;
  - value = cost × max(0, 1 − months / category life), rounded to paise;
  - a category life of 0 means no depreciation.
- **Due lists:**

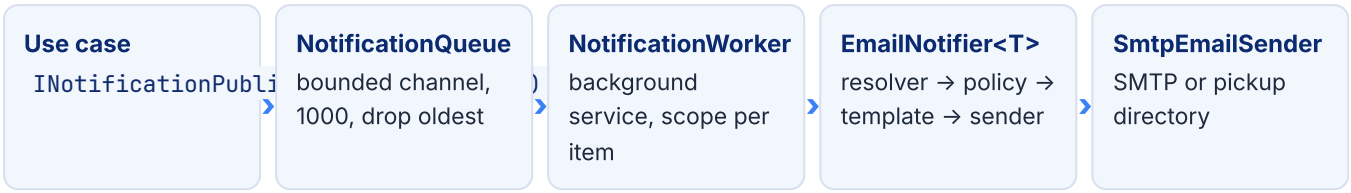
    - audits due: next audit within 30 days;
    - warranty ending: warranty end within the next 90 days;
    - overdue: checked out with an expected check-in date before today;
    - licences expiring (digest): within 30 days.
- **Dashboard and reports** cover only the assets the caller may view (the same filter as lists).
- **CSV:**
    - cells starting with `= + - @` get a leading apostrophe (formula-injection protection);
    - cells containing a comma, quote or newline are quoted;
    - the file is UTF-8 with a BOM, for Excel.

9 AMS data layer: Dapper and stored procedures (L-38, L-39)

- `DapperHelper`:
  - opens a connection per call and runs a stored procedure;
  - logs queries slower than **3 s** and SQL errors with the procedure name and parameters (secrets masked), then rethrows;
  - methods: `ListAsync`, `SingleAsync`, `MultipleAsync` (several result sets), `WriteAsync`, `ExecuteBatchAsync`.
- **Write contract.** Every write procedure has outputs `@Error_Code INT`, `@Error_Message NVARCHAR` and, when inserting, `@Record_Id`. 0 = success, 1001 = not found, 1002 = rule violation, 1003 = duplicate. `DbResult.EnsureOk()` turns a failure into an `AmsException` with the same code.
- **Tenant isolation.** Every table has `TenantId`; every procedure takes `@TenantId` from the token. A row of another tenant is "not found".
- **Audit fields.** Writes receive `@ActorEmpCode`, `@ActorName` and `@Now` (IST business time) and add a `History` row (item type, id, action, target, note, location).
- **Enums** are passed and stored as strings (readable in SQL).
- **Schema:** 15 tables:
  - setup: Categories, Manufacturers, Suppliers, Locations, Departments, StatusLabels, AssetModels;
  - assets: Assets, Maintenances, AssetRequests;
  - licences: Licenses, LicenseSeats;
  - stock: StockItems, StockAssignments;
  - History.

  Plus the view `vw_Assets` and 34 procedures named `PRO_AMS_<AREA>_<ACTION>`.
- **Initialisation** (development, `AutoMigrate`):
  - create the database if missing;
  - run `001_ams_schema.sql` once (only when `dbo.Assets` does not exist);
  - always re-run `002_ams_procedures.sql` (CREATE OR ALTER), split on `GO`;
  - if `DemoTenantId` is set and the database is empty, write demo data through the same procedures.

10 Notifications (L-40 – L-42)



- **Separation** (SRP):
  - `IRecipientResolver<T>` decides who receives a message;
  - `IRecipientPolicy` drops invalid addresses, de-duplicates and keeps only `AllowedDomains`;
  - `IEmailTemplate<T>` builds the subject and the HTML body (values HTML-encoded);
  - `IEmailSender` sends.

  A new e-mail = a model + a template + a resolver; nothing else changes.
- **Failure isolation.** Sending runs after the transaction, in the background. A mail failure is logged and never fails the user's action.
- **Daily digest.** `DailyDigestService` wakes at `DailyDigestTime` (IST) and builds, per tenant:
  - overdue check-ins;
  - audits due within 30 days;
  - licences expiring within 30 days;
  - low stock.

  It sends to `AssetDeskEmail`, and is skipped when there is nothing to report.
- **People directory.** `UmsPeopleDirectory` searches UMS `me/directory` and caches results per tenant and search for 2 minutes.

11 In-house building blocks (L-43, L-44)

11.1 VMM.Mediator

- **Contracts:** `IRequest<T>`, `IRequestHandler<TReq,TRes>`, `IPipelineBehavior<TReq,TRes>`, `ISender` / `IPublisher` / `IMediator`, `INotificationHandler<T>`, `Unit`.
- **Send:**
  - resolves the handler and all behaviours for the request type from DI;
  - behaviours wrap the handler in registration order (first registered = outermost);
  - the generic pipeline per request type is built once and cached, so no reflection runs per call;
  - a missing handler raises `HandlerNotFoundException`.
- **Publish:** runs every `INotificationHandler<T>` of the event's type and of its base classes, in order; no handler is fine.
- **Registration:** `AddMediator(cfg ⇒ cfg.RegisterServicesFromAssembly(..).AddOpenBehavior(..))` scans assemblies for handlers.

11.2 VMM.Mapper

- **Profiles:** a `MapperProfile` declares `CreateMap<TSource,TDest>()`, with `ForMember`, `Ignore`, `ConstructUsing` and `ReverseMap`.
- **Plan per map, compiled once:**
  - pick a factory: the given factory, else a parameterless constructor, else the public constructor with the most parameters that can all be filled (records);
  - then assign the writable properties not set by the constructor;
  - getters are compiled expression trees.
- **Conversions:** assignable types, nullable, enum ↔ string / number, IConvertible, nested maps, collections (List, array, IReadOnlyList…).
- **Validation:** `AssertConfigurationIsValid()` lists every destination member nothing maps to, and every type that cannot be converted. `AddObjectMapper(assemblies)` runs it at start-up.

12 Configuration reference

Component	Key	Meaning
UMS API	`Persistence:Provider`, `ConnectionStrings:Default`	InMemory (demo) or SqlServer
UMS API	`Jwt:Issuer / Audience / Secret`	Token signing; secret ≥ 32 chars from a secret store
UMS API	`Security:RequireAuthentication`	true in production
UMS (per tenant)	Security Settings	SessionTimeoutMinutes, SingleDeviceLogin, MaxFailedLogins, LockoutMinutes, PasswordMinLength, JwtExpiryMinutes, RefreshTokenDays
AMS.Api	`ConnectionStrings:Ams`, `Ams:Database:AutoMigrate / DemoTenantId`	Database; dev-only auto-migration and seed
AMS.Api / Portal	`Ums:BaseUrl / AppCode / Device / Channel / JwtIssuer / JwtAudience / JwtSecret / TrustedForwarders`	UMS connection and token validation
AMS.Api	`Notifications:*`, `EmailServerSettings:*`	From, AssetDeskEmail, PortalUrl, AllowedDomains, DailyDigestEnabled/Time; SMTP or PickupDirectory
AMS.Portal	`AmsApi:BaseUrl / TimeoutSeconds / Routes`	API location and route overrides
AMS.Portal	`Auth:RevalidationSeconds / CookieLifetimeMinutes / AbsoluteLifetimeHours`	60 / 60 / 12
All	`Logging:FilePath`	Serilog rolling files per level

13 Tests

Project	Tests	Covers
UMS.Tests	45	Decision engine (layers, scopes, policies, schedules, holidays, grants), storage formats, sessions (single device, refresh rotation, reuse, idle, lockout, validator end-recording), AMS policies.
AMS.Tests	29	Use cases with fakes (UMS asked first; deny writes nothing; return date; procedure errors; row actions; approval needs checkout; rejection reason); ApiCallService; UmsAccessGuard (fail closed, session end, network); recipient policy; depreciation; CSV. 4 SQL Server integration tests (tenant isolation, double check-out, round trip with history, unique tags) run when `AMS_TEST_SQL` is set.
BuildingBlocks.Tests	14	Mediator dispatch, Unit, behaviour order, container behaviours, missing handler, events with base types; mapper records, conversions, nested and collections, ignore, copy onto object, reverse, validation, registration.

Run: `dotnet test UMS_Sol.sln`. End-to-end behaviour was verified in a browser:

- role views;
- single-device kick-out;
- idle sign-out;
- token refresh with 1-minute tokens;
- admin revoke;
- request → approve → issue, with e-mails.